Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	13 February 2026
Team ID	LTVIP2026TMIDS81332
Project Name	flightfinder: navigating your air travel options
Maximum Marks	4 Marks

Technical Architecture:

For a modern, scalable flight app, the MERN Stack (which you are already using) is the industry standard. Here is the technical justification for each layer:

3.1 Frontend: React.js

- **Why: React uses a Virtual DOM, which is essential for updating flight prices and filters instantly without refreshing the page.**
- **Libraries: Redux for managing the state of the "Selected Flight," and Tailwind CSS for a responsive, mobile-first design.**

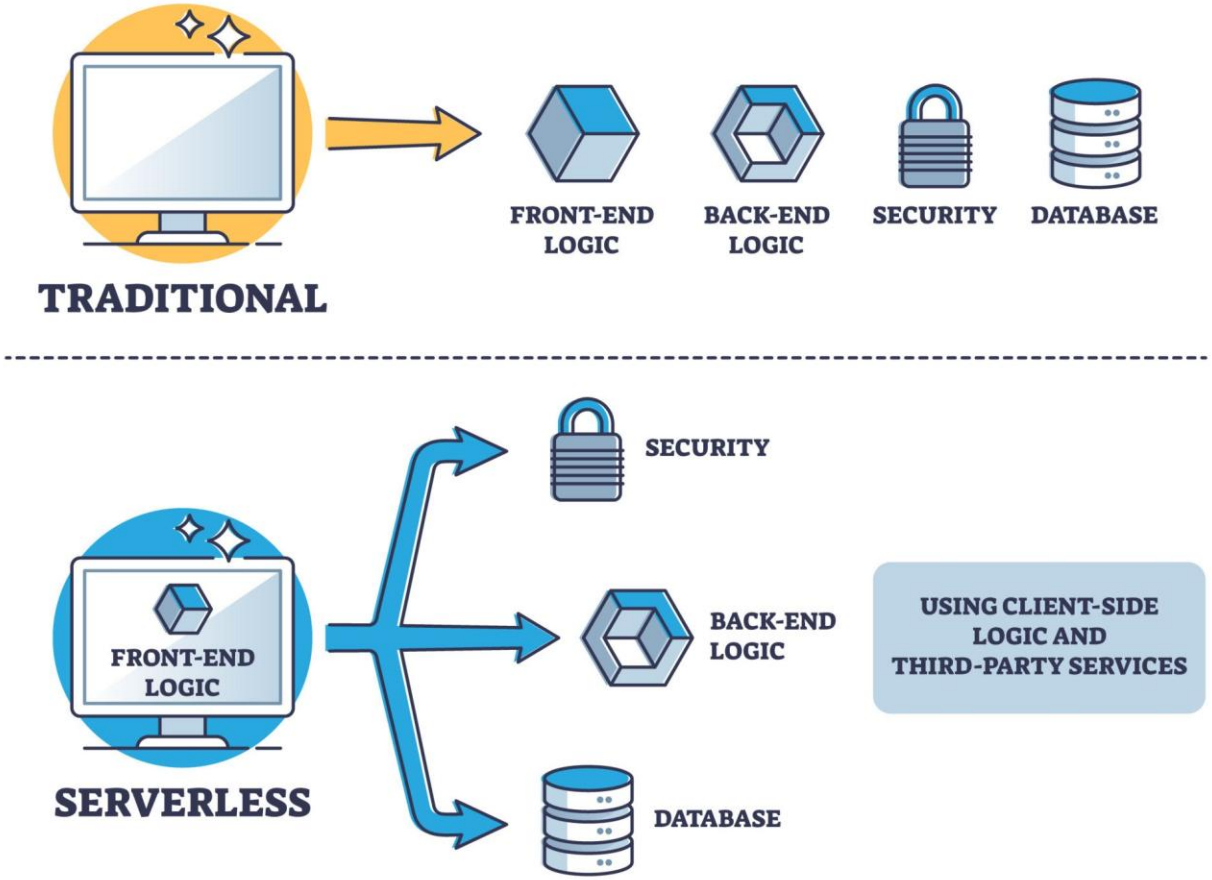
3.2 Backend: Node.js & Express.js

- **Why: Node.js is non-blocking and asynchronous. This is critical because fetching flight data from external APIs can take time; Node allows the server to handle other requests while waiting for the API to respond.**
- **Middleware: Cors for cross-origin requests and Dotenv for securing your MongoDB URI and API keys.**

3.3 Database: MongoDB Atlas (NoSQL)

- **Why: Flight data is often "nested" (e.g., a flight has multiple segments, which have multiple seat types). A document-based database like MongoDB handles this hierarchical data much better than a traditional SQL table.**
- **Mongoose: Used as the Object Data Modeling (ODM) library to enforce a schema on top of the flexible MongoDB documents.**

3.4 DevOps & APIs

- **External API: Amadeus for Developers or RapidAPI (Skyscanner) for real-time flight data.**
- **Payment: Stripe API for secure, encrypted payment processing.**
- **Deployment: Vercel for the Frontend and Render or Heroku for the Backend.**

TRADITIONAL VS SERVERLESS ARCHITECTURE